

UNIVERSIDADE DE VASSOURAS

Campus Maricá

Curso de Graduação em Engenharia de Software

PORTAL DE ÚLTIMAS NOTÍCIAS DE MARICÁ

Documentação Técnica do Projeto — Entrega P1

Sistema web desenvolvido em Django com fluxo de publicação e aprovação editorial

Kariny Cunha do Nascimento

Matrícula: 202423500

Disciplina: Laboratório de Programação Full Stack

Professor: Márcio Garrido

Maricá — RJ

2026

Sumário

1. Introdução
2. Objetivos
3. Tecnologias Utilizadas
4. Arquitetura do Sistema
5. Modelagem de Dados
6. Funcionalidades Implementadas
7. Segurança
8. Como Executar o Projeto
9. Considerações Finais e Próximos Passos (P2)
10. Referências

1. Introdução

Este documento apresenta a documentação técnica do projeto "Portal de Últimas Notícias de Maricá", desenvolvido como entrega P1 (MVP) da disciplina Laboratório de Programação Full Stack, do curso de Engenharia de Software da Universidade de Vassouras, Campus Maricá.

O projeto consiste em um sistema web de blog/portal de notícias, construído em Django puro (sem framework de front-end separado), no qual o próprio servidor monta e entrega as páginas HTML através de templates. O sistema permite o cadastro de matérias jornalísticas, organizadas por categorias e tags, com autoria identificada, espaço para comentários dos leitores e, principalmente, um fluxo de publicação com aprovação editorial — o requisito central desta etapa do trabalho.

O tema "Blog / Portal de Notícias" foi escolhido dentre a lista de projetos apresentada pela disciplina, cujo escopo do P1 previa a implementação das entidades Post, Categoria, Tag, Autor e Comentário, além de um fluxo de publicação com no mínimo dois estados distintos.

2. Objetivos

2.1 Objetivo Geral

Desenvolver um sistema web completo de portal de notícias utilizando o framework Django, aplicando os conceitos de modelagem de dados, ORM, views, URLs, templates, formulários e o padrão arquitetural MVT (Model-View-Template), estudados ao longo do bloco de fundamentos da disciplina.

2.2 Objetivos Específicos

- Modelar as entidades do domínio (Post, Categoria, Tag, Autor e Comentário) e seus relacionamentos;
- Implementar um fluxo de publicação editorial com múltiplos estados (Rascunho, Em Revisão e Publicado), simulando o processo real de aprovação de conteúdo em um veículo de notícias;
- Construir uma interface pública para leitura das matérias, navegação por categoria e envio de comentários;
- Construir um painel administrativo interno (Painel Editorial) para gestão do ciclo de vida das matérias;
- Aplicar boas práticas de segurança básica (proteção CSRF, variáveis de ambiente para credenciais);
- Produzir uma interface visual coerente, responsiva e com identidade própria para o portal.

3. Tecnologias Utilizadas

O projeto foi desenvolvido com as seguintes tecnologias e bibliotecas:

- Python 3.14 — linguagem de programação do back-end;
- Django 5.2 — framework web utilizado para toda a aplicação (models, views, templates, admin, forms);
- PostgreSQL — banco de dados relacional (com SQLite disponível como alternativa local de desenvolvimento);

- HTML5 e CSS3 — estruturação e estilização das páginas, escritos manualmente (sem framework de CSS);
- Django Template Language (DTL) — montagem dinâmica do HTML a partir dos dados do banco;
- python-dotenv — carregamento de credenciais e configurações sensíveis a partir de variáveis de ambiente;
- Pillow — manipulação de imagens (necessária para o campo de imagem de capa das matérias);
- Google Fonts (Montserrat e Playfair Display) — tipografia customizada da identidade visual do portal;
- Git — controle de versão do código-fonte.

4. Arquitetura do Sistema

4.1 Padrão MVT (Model-View-Template)

O Django organiza a aplicação segundo o padrão MVT, uma variação do tradicional MVC. Nesse padrão:

- Model — define os dados e as regras de negócio, mapeados para tabelas do banco via ORM;
- View — função Python que recebe a requisição HTTP, consulta os models necessários e decide qual resposta devolver;
- Template — arquivo HTML com marcações especiais (DTL) que a view usa para montar a página final enviada ao navegador.

O ciclo de uma requisição no projeto segue sempre o caminho: URL → View → Model (quando necessário) → Template → Resposta HTTP.

4.2 Organização do Projeto e do App

Seguindo a convenção do Django, o código está dividido em um projeto (configurações gerais) e um app (a funcionalidade de notícias em si):

- portal_noticias/ — pacote de configuração do projeto (settings.py, urls.py, wsgi.py, asgi.py);
- noticias/ — app responsável por toda a regra de negócio do portal, contendo models.py, views.py, urls.py, forms.py, admin.py, a pasta templates/noticias/ (com herança de template via base.html), a pasta static/noticias/ (CSS e imagens) e o comando personalizado seed_dados.py, usado para popular o banco com dados de exemplo.

5. Modelagem de Dados

O sistema é composto por cinco entidades principais, implementadas como classes Python (models) que o Django traduz automaticamente em tabelas do banco de dados através do ORM e do sistema de migrações. A seguir, o detalhamento de cada uma delas.

5.1 Categoria

Representa o assunto/editoria de uma matéria (ex.: Política, Tecnologia, Esportes).

Campo	Tipo	Descrição
nome	CharField	Nome da categoria (único).

5.2 Tag

Palavra-chave livre que pode ser associada a várias matérias, usada para classificação complementar.

Campo	Tipo	Descrição
nome	CharField	Nome da tag (único).

5.3 Autor

Representa o jornalista/redator responsável pela matéria.

Campo	Tipo	Descrição
nome	CharField	Nome completo do autor.
email	EmailField	E-mail de contato do autor.
bio	TextField	Breve biografia (campo opcional).

5.4 Post

Entidade central do sistema: representa uma matéria/notícia, com o campo "status" responsável pelo fluxo de publicação editorial (detalhado na Seção 6.1).

Campo	Tipo	Descrição
titulo	CharField	Título da matéria.
resumo	CharField	Texto curto exibido na listagem.
conteudo	TextField	Corpo completo da matéria.
imagem_capa	ImageField	Imagem de destaque (opcional).
categoria	FK → Categoria	Categoria à qual a matéria pertence.
autor	FK → Autor	Autor responsável pela matéria.
tags	M2M → Tag	Palavras-chave associadas (opcional).
status	CharField (choices)	Rascunho, Em Revisão ou Publicado.
data_criacao	DateTimeField	Preenchida automaticamente na criação.
data_atualizacao	DateTimeField	Atualizada automaticamente a cada alteração.
data_publicacao	DateTimeField	Preenchida no momento da aprovação/publicação.
visualizacoes	PositiveIntegerField	Contador de acessos, usado no ranking "Mais Lidas".

5.5 Comentário

Mensagem deixada por um leitor em uma matéria específica.

Campo	Tipo	Descrição
post	FK → Post	Matéria à qual o comentário pertence.
nome_autor	CharField	Nome informado pelo leitor.
email	EmailField	E-mail informado pelo leitor.
texto	TextField	Conteúdo do comentário.
data_criacao	DateTimeField	Preenchida automaticamente na criação.

5.6 Relacionamentos

- Uma Categoria pode estar associada a várias matérias (1:N);
- Um Autor pode assinar várias matérias (1:N);
- Uma matéria pode ter várias Tags, e uma Tag pode estar em várias matérias (N:N);
- Uma matéria pode receber vários comentários (1:N).

6. Funcionalidades Implementadas

6.1 Fluxo de Publicação com Aprovação Editorial

O requisito central do P1 é o fluxo de publicação com no mínimo dois estados. O projeto implementa três estados, superando o mínimo exigido:

- Rascunho — estado inicial de toda matéria nova; não é exibida no site público;
- Em Revisão — a matéria foi enviada pelo redator e aguarda aprovação;
- Publicado — a matéria foi aprovada, recebe a data de publicação e passa a ser exibida publicamente.

As transições de estado são feitas por ações específicas no Painel Editorial: enviar para revisão, aprovar e publicar, devolver para rascunho (quando o revisor pede ajustes) e despublicar (retirar do ar uma matéria já publicada). Cada transição é implementada como uma view própria em `views.py`.

6.2 Área Pública

- Página inicial com a listagem de todas as matérias publicadas, em formato de cards;
- Listagem filtrada por categoria;
- Página de detalhe de cada matéria, com conteúdo completo, tags e imagem de capa (quando cadastrada);
- Formulário de comentários na página de detalhe, com validação e proteção CSRF.

6.3 CRUD de Matérias

Implementado através de um `ModelForm` (`PostForm`), o sistema permite criar, editar e excluir matérias, com confirmação antes da exclusão. O campo de status é controlado apenas pelas

ações do fluxo editorial, nunca digitado livremente no formulário.

6.4 Painel Editorial

Tela interna que lista todas as matérias do sistema, independentemente do status, com indicação visual (selo colorido) do estado atual e os botões de ação disponíveis para cada uma, de acordo com sua etapa no fluxo de publicação.

6.5 Contador de Visualizações e Ranking "Mais Lidas"

Cada acesso à página de uma matéria publicada incrementa o campo visualizacoes. Esse dado alimenta um widget de "Mais Lidas" na barra lateral do site, ordenado pelo número real de acessos — não se trata de um dado estático ou ilustrativo.

6.6 Layout e Identidade Visual

- Cabeçalho com fotografia de Maricá como plano de fundo e logotipo próprio ("Portal Notícia"), combinando as fontes Montserrat e Playfair Display;
- Layout de duas colunas na página inicial e nas páginas de categoria: coluna principal com as notícias e barra lateral com widgets úteis (Mais Lidas, Previsão do Tempo ilustrativa e lista de Categorias);
- Rodapé estruturado em colunas, com informações institucionais, redes sociais, links rápidos, seção "Sobre Nós" e informações de contato;
- Layout responsivo, adaptado para exibição em telas menores.

6.7 Django Admin

Todos os models estão registrados no Django Admin, com filtros, campos de busca e listagem configurados, permitindo o cadastro rápido de categorias, tags e autores antes da criação das matérias.

7. Segurança

- Proteção CSRF em todos os formulários (tag {% csrf_token %}), impedindo o envio de dados de origem não confiável;
- Uso do ORM do Django em todas as consultas, o que evita a escrita manual de SQL e reduz o risco de SQL Injection;
- Credenciais sensíveis (senha do banco de dados, chave secreta do Django) mantidas fora do código-fonte, em um arquivo .env carregado pela biblioteca python-dotenv e excluído do controle de versão pelo .gitignore;
- Autenticação e controle de acesso por papéis de usuário ainda não implementados nesta etapa — ver Seção 9 (Considerações Finais).

8. Como Executar o Projeto

Pré-requisitos: Python 3.10 ou superior instalado.

8.1 Passo a Passo

- Criar e ativar o ambiente virtual: `python -m venv venv` e `.\venv\Scripts\activate` (Windows);

- Instalar as dependências: `pip install -r requirements.txt`;
- Configurar o arquivo `.env` (já incluso um exemplo em `.env.example`);
- Aplicar as migrações do banco de dados: `python manage.py migrate`;
- (Opcional) Popular o banco com dados de exemplo: `python manage.py seed_dados`;
- Criar um usuário administrador: `python manage.py createsuperuser`;
- Iniciar o servidor de desenvolvimento: `python manage.py runserver`;
- Acessar `http://127.0.0.1:8000/` no navegador (o Django Admin fica em `/admin/`).

9. Considerações Finais e Próximos Passos (P2)

O projeto atende integralmente ao escopo definido para o P1: as cinco entidades solicitadas foram modeladas, o fluxo de publicação com aprovação editorial foi implementado com três estados (superando o mínimo de dois exigidos), e a aplicação está funcional de ponta a ponta, seguindo o padrão MVT e as boas práticas ensinadas ao longo do bloco de fundamentos da disciplina.

A segunda etapa do trabalho (P2), prevista para as aulas finais do cronograma da disciplina, prevê a evolução deste mesmo projeto com os seguintes pontos, ainda não implementados:

- Múltiplos blogs — suporte a mais de um portal/blog dentro do mesmo sistema;
- Papéis de usuário — autenticação e permissões diferenciadas (administrador, editor, redator, leitor);
- Dashboard — painel com métricas e indicadores visuais de desempenho do portal;
- API REST — exposição dos dados via Django REST Framework (DRF), permitindo integração com outras aplicações.

10. Referências

DJANGO SOFTWARE FOUNDATION. *Django Documentation*. Disponível em: <https://docs.djangoproject.com/>. Acesso em: 2026.

GARRIDO, Márcio. *Material de aula da disciplina Laboratório de Programação Full Stack*. Universidade de Vassouras, Campus Maricá, 2026.

PYTHON SOFTWARE FOUNDATION. *Python Documentation*. Disponível em: <https://docs.python.org/>. Acesso em: 2026.